TutorPro — Project Handoff & Offboarding

Oct 31, 2026 · @Armin

Summary

TutorPro is live and feature-complete: a website plus booking system for a private math and physics tutor (university, IB, AP, SAT, A-Level). It is deployed and verified in production as of 2026-10-25; the remaining work is content entry, one real-world booking test, and key rotation.

What it does:

- **Markets the tutor** — home, about, courses, exam prep, policy, reviews and free sample resources, all editable from the admin dashboard.
- **Takes students from first contact to a booked session** — contact/session request → tutor accepts → student books through Calendly → emails at each step.
- **Runs the practice** — the tutor manages requests, bookings, student records, course assignments, a resource library and every page of site content from `/admin`.

Thing	Where
Live website	tutoring-website-ram-frontend.vercel.app
Live API	tutorpro-backend-production.up.railway.app (`/health` shows which integrations are configured)
Code	github.com/RamtinTutorWeb/Tutoring-Website-Ram, branch `main` (commit `33d7ffc` is what's deployed)
Latest feature PR	#8 — resource library, student records, editable assessment, photo upload, e2e suite
In-repo docs	`README.md`, `docs/ARCHITECTURE.md`, `docs/DEPLOY.md`, `supabase/README.md`

Offboarding checklist

The new owner needs admin access to seven services; nothing lives only on the outgoing developer's laptop except the local `.env` files (which are copies of what's in Railway/Vercel).

Accounts to hand over

Service	What's there	Identifier	Transfer how
GitHub	Source code, keepalive workflow	Org `RamtinTutorWeb`, repo `Tutoring-Website-Ram`	Add new owner as org Owner
Vercel	Frontend hosting + `VITE_*` env vars	Project `tutoring-website-ram-frontend`	Invite to the Vercel team, or transfer project
Railway	Backend API + secret env vars	Workspace "Ramtin Gorji's Projects", project `generous-wisdom`, service `tutorpro-backend`, env `production`	Invite to workspace as Admin
Supabase	Postgres database + file storage	Project ref `dgnyvwxodajmkfaevpjn`	Invite to the Supabase org as Owner
Clerk	User accounts, sign-in, admin role	TutorPro application (currently the **development** instance, `pk_test_` keys)	Invite to the Clerk application as Admin
Resend	Transactional email	Sending as `onboarding@resend.dev` (no own domain yet)	Invite to Resend team
Calendly	Booking pages, booking webhooks	The tutor's Calendly account	Stays with the tutor
Cloudflare Turnstile	Optional contact-form spam check	Not set up yet	—

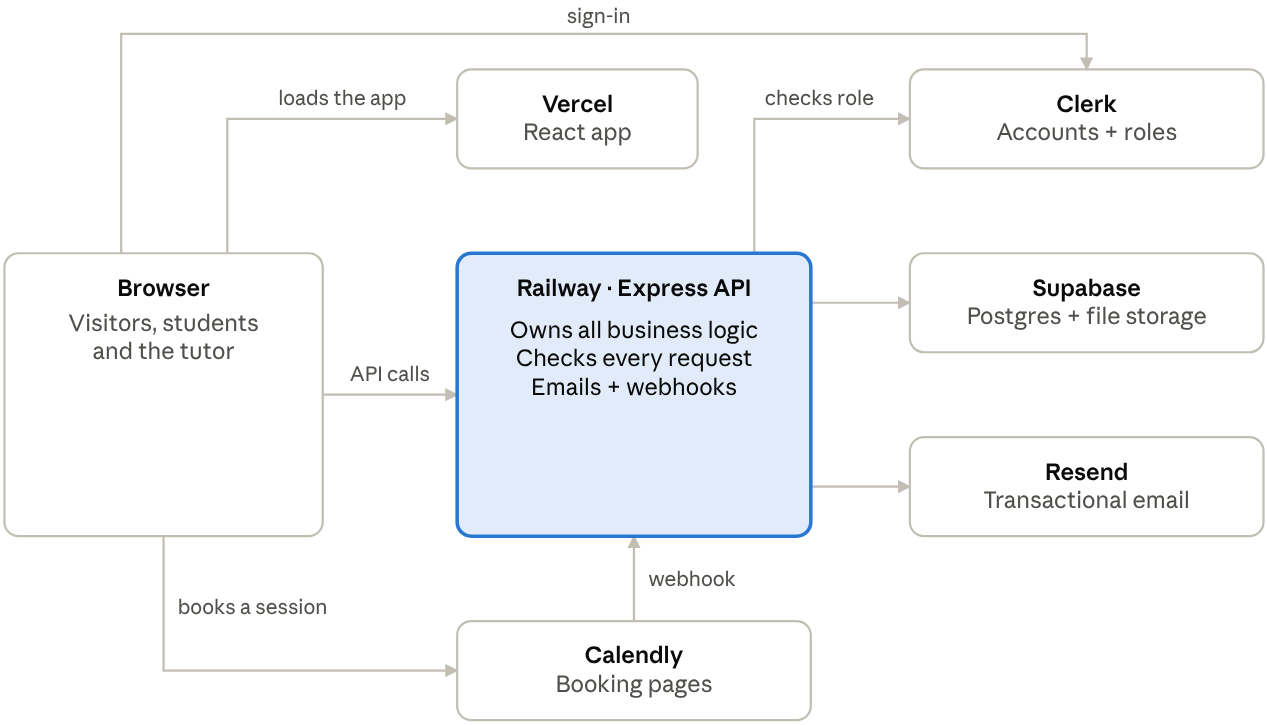
Handover steps

- [ ] Grant the new owner access to every service above, and confirm they can log in to each.
- [ ] Rotate every backend secret (Railway had a platform security incident on 2026-10-21 and advised all customers to rotate): `CLERK_SECRET_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `RESEND_API_KEY`, `CALENDLY_WEBHOOK_SIGNING_KEY`, `ADMIN_API_TOKEN` (and the Calendly personal access token if one was created). Paste new values into Railway → tutorpro-backend → Variables; Railway redeploys.
- [ ] After rotating `CALENDLY_WEBHOOK_SIGNING_KEY`, re-register the Calendly webhook (see Deployment).
- [ ] Remove the outgoing developer from each service once the new owner is confirmed.
- [ ] Make the tutor's Clerk user an admin: Clerk → Users → user → Public metadata `{ "role": "admin" }`.
- [ ] Clone the repo fresh outside iCloud (the old copy on the Desktop has a corrupted git pack because iCloud offloaded it).
- [ ] Close the production config gaps listed under Known issues (email domain, `ADMIN_EMAIL`, Clerk webhook, Clerk production instance).

Architecture

The browser loads a static React app from Vercel and sends every data request to one Express API on Railway; that API is the only thing allowed to touch the database, send email or receive webhooks.

The browser talks only to the API; only the API touches the database



system architecture · 7 services

The one direct browser-to-Supabase call is a resource download: the API checks access, then hands out a 5-minute signed Storage link.

Component	Tech	Responsibility
Frontend	React 18, Vite 7, react-router 6, `@clerk/react`, `react-calendly`	Pages, admin UI, Clerk sign-in widget, Calendly embed
Backend	Node 22, Express 5, TypeScript, zod 4, `@clerk/express`, `@supabase/supabase-js`, Resend SDK	Validation, authorization, all reads/writes, email, webhooks
Database	Supabase Postgres + Storage	Data and files; locked to the service role
Auth	Clerk	Accounts, sessions, admin role in public metadata
Booking	Calendly	Event types and scheduling UI; webhooks report bookings and cancels

Component	Tech	Responsibility
Email	Resend	New-request, request-accepted and resource-shared emails
Spam check	Cloudflare Turnstile (optional)	Bot check on the contact form

Repository layout

One npm-workspaces monorepo (Node 22+): `frontend/` and `backend/` are separate packages; Supabase migrations and Playwright tests sit at the root.

Path	What it is
`package.json`	Workspace root; scripts `dev:frontend`, `dev:backend`, `build`, `typecheck`, `test`, `e2e`
`vercel.json`	Vercel build (frontend workspace only), SPA rewrite, security headers, immutable asset caching
`railway.json`	Railway build/start commands, `/health` healthcheck, restart policy, watch paths
`.github/workflows/keepalive.yml`	Daily `GET /content` so the free Supabase project never pauses
`docs/ARCHITECTURE.md`	HTTP contract, env vars, shared types — the authoritative API spec
`docs/DEPLOY.md`	Step-by-step account setup for every service
`supabase/migrations/`	5 SQL migrations (schema, lockdown, reviews, resources, site-media bucket)
`supabase/seed.sql`	Local dev data only — never run in production
`supabase/config.toml`	Local stack (Postgres + PostgREST + Studio; Supabase Auth disabled)
`backend/src/app.ts`, `server.ts`	Express app factory and process entry
`backend/src/env.ts`	Reads env vars; `configured()` drives `/health` and graceful degradation

Path	What it is
`backend/src/db.ts`	Every database/storage call (supabase-js, service role); snake_case ↔ camelCase
`backend/src/routes/`	One file per resource: content, requests, bookings, learnerCourses, reviews, assessments, resources, media, me, admin, webhooks, health
`backend/src/lib/`	`clerk.ts` (role lookup, 60 s cache), `mail.ts` (Resend templates), `calendly.ts`, `content.ts` (defaults + normalizers), `turnstile.ts`, `supabase.ts`
`backend/src/middleware/auth.ts`	`requireAuth` / `requireAdmin`, Clerk session + DEV_AUTH
`backend/test/`	vitest + supertest suites with in-memory fakes (`fakes.ts`) for DB, Clerk and mail
`frontend/src/router/`	Route table (`AppRouter`)
`frontend/src/api/`	`client.ts` (fetch + bearer token), `hooks.ts`, `ContentProvider` (site content), `MeProvider` (current user), `types.ts`
`frontend/src/auth/`	Clerk vs dev-auth session switch, `RequireAuth` / `RequireAdmin` guards, sign-in pages
`frontend/src/pages/`	Public pages, `dashboard/` (student), `admin/` (tutor dashboard, page editors, student record)
`frontend/src/components/`	Calendly embed, resource library/manager, Turnstile, review form, video player, etc.
`frontend/src/styles.css`	All styling (CSS variables, Inter font, brand `#2f6bff`)
`frontend/vite.config.ts`	Includes the SEO plugin that writes `robots.txt` / `sitemap.xml` from `VITE_SITE_URL`
`e2e/`	Playwright config, helpers and `tutor-app.spec.ts` (9 end-to-end scenarios)

Data model

All data lives in one Supabase Postgres database with nine tables and two storage buckets; only the backend can reach it. Every table has Row Level Security on with **no policies**, and the `anon` / `authenticated` roles hold no privileges, so a leaked anon key reads nothing. The backend's service-role key bypasses RLS.

Tables

Table	Purpose	Notable columns / rules
`profiles`	One row per Clerk user	`id` = Clerk user id; `role` is a **mirror** of Clerk metadata, never used for authorization. Written by `GET /me` (upsert) and the Clerk webhook
`site_settings`	Editable site content as JSON blobs by key	`content` (courses, examPrepTracks, faq, assessmentQuestions), `selectable_options` (dropdowns), `pages` (home/about/examPrep/policy/contact/booking). `session_settings` is legacy, unused
`reviews`	Testimonials	`status` `pending` (student-submitted) or `approved` (public)
`session_requests`	Contact-form / session requests	`student_id` null for guests; `status` new → accepted/declined → scheduled → closed
`bookings`	Mirror of Calendly invitees	unique on `calendly_invitee_uri`; linked to a request via Calendly `utm_content`; `cancel_url`, `reschedule_url`; `status` scheduled/canceled
`assessments`	Placement test results	`score`, `total`, `recommendation`, raw `answers`
`learner_courses`	A student's enrolled courses	unique (student, course); `status` registered / in-progress / passed
`resources`	Resource library	`kind` file/link/video; `visibility` public/students/private; `student_ids text[]` for private shares; `featured` shows on home page; check constraint: files need `storage_path`, links need `url`

Table	Purpose	Notable columns / rules
`webhook_events`	Idempotency ledger	unique `(provider, event_id)` so Clerk/Calendly retries are processed once

Storage buckets

Bucket	Access	Limit	Used for
`resources`	Private; files served only via 5-minute signed URLs from the backend	50 MB per file	Resource library files
`site-media`	Public read; only the backend writes	5 MB, PNG/JPEG/WebP/GIF/AVIF	Tutor photo and other page images

Migrations

File	Change
`20261022000000_site_media.sql`	Public `site-media` bucket
`20261021000000_resources_booking_links.sql`	`resources` table + private bucket; booking cancel/reschedule URLs (with backfill)
`20260928000000_reviews_table_group_bookings.sql`	Reviews moved to their own table; group-event bookings
`20260927000000_backend_only.sql`	Locks out anon/authenticated, contact-form requests, assessment totals, default dropdowns
`20260903000000_init.sql`	Initial schema

All five are applied to production. Every migration is written to be safe to re-run. New tables must enable RLS and grant only `service_role`.

Auth and roles

Clerk handles every sign-in; the backend verifies the Clerk session token on each request and reads the role from Clerk, never from its own database.

Roles

Role	How you get it	Can do
Visitor	Not signed in	Browse every public page, open public resources, send a contact/session request, take the assessment (result not saved)
Student	Any signed-up Clerk user (default)	Visitor rights + students-only and privately shared resources, saved assessments, own requests and bookings (with Calendly reschedule/cancel links), register/drop courses, submit reviews (go to moderation), edit own name/phone
Admin	Clerk → Users → user → **Public metadata** `{ "role": "admin" }`	Everything: accept/decline requests, all bookings, per-student records, resource upload and sharing, course assignment and progress, review moderation, editing all site content and assessment questions

How a request is authenticated

1. The frontend (`frontend/src/auth/session.tsx`) gets a session token from Clerk and sends it as `Authorization: Bearer <token>` (`frontend/src/api/client.ts`).
2. `clerkMiddleware` (`backend/src/lib/clerk.ts`) verifies it offline via JWKS. In production it only accepts tokens minted for origins listed in `FRONTEND_URL` (authorized parties).
3. `requireAuth` / `requireAdmin` (`backend/src/middleware/auth.ts`) look up the Clerk user and its `publicMetadata.role` (cached 60 s per user, so a role change takes up to a minute). Anything other than `"admin"` is a student.

4. `GET /me` upserts the user's `profiles` row so new users appear in the admin student list even if the Clerk webhook never fires.

Other auth mechanisms

- **`x-admin-token` header** — guards maintenance routes under `/admin/calendly/*`; compared in constant time to `ADMIN_API_TOKEN`.
- **Webhook signatures** — Clerk webhooks are verified with svix (`CLERK_WEBHOOK_SIGNING_SECRET`); Calendly webhooks with HMAC (`CALENDLY_WEBHOOK_SIGNING_KEY`).
- **DEV_AUTH (local only)** — `DEV_AUTH=1` on the backend and `VITE_DEV_AUTH=1` on the frontend replace Clerk with "Tutor (admin)" / "Student" buttons, using tokens like `Bearer dev:user_dev_admin`. The backend refuses it unless Clerk keys are empty, `NODE_ENV` isn't `production` **and** `SUPABASE_URL` is localhost; production frontend builds strip the code entirely (verified: absent from the live bundle).

Backend API reference

The API is JSON over HTTPS, camelCase in and out, with errors as `{ "error": "..." }` and status 400/401/403/404/409/429/500/503. `docs/ARCHITECTURE.md` holds the full TypeScript types; this table is the access map.

Method	Path	Access	What it does
GET	`/health`	public	`{ ok, configured: { clerk, supabase, resend, calendly, turnstile } }`
GET	`/content`	public (auth optional)	All site content; pending reviews only for admins
PUT	`/content`	admin	Partial update; each key present replaces the stored value, each page in `pages` replaces that page. Rejects `reviews` (400)

Method	Path	Access	What it does
GET / PATCH	`/me`	auth	Current profile (upserted from Clerk on first call) / edit `fullName`, `phone`
POST	`/requests`	public	New contact/session request; emails `ADMIN_EMAIL`. Rate limit 10 per 15 min per IP; Turnstile token required when configured
GET	`/requests`	auth	Student: own; admin: all, newest first
PATCH	`/requests/:id`	admin	Set `status`; `accepted` emails the student a `/book?request=<id>` link
GET	`/bookings`	auth	Student: own; admin: all, by start time
GET / POST	`/learner-courses`	auth	List / register a course (admin may pass `studentId` to assign). 400 for unknown course, 409 duplicate
PATCH	`/learner-courses/:id`	admin	Set progress `registered` / `in-progress` / `passed`
DELETE	`/learner-courses/:id`	auth	Admin any, student own
POST	`/reviews`	auth	Student → `pending`, admin → `approved`; rate-limited, 429 when too many pending
PATCH / DELETE	`/reviews/:id`	admin	Moderate / edit / delete
GET / POST	`/assessments`	auth	List (own / all) / save a result

Method	Path	Access	What it does
GET	`/resources`	public (auth optional)	Filtered by visibility; `studentIds` only shown to admin
GET	`/resources/:id/download`	public (auth optional)	`{ url }` — 5-minute signed link for files, target URL for links/videos; hidden = 404, `no-store`
POST	`/resources/files?name=`	admin	Raw file bytes (own Content-Type) → storage path; step 1 of upload
POST / PATCH / DELETE	`/resources[/:id]`	admin	Create / edit / delete; newly added private recipients are emailed; replaced or deleted files are removed from storage
POST	`/media?name=`	admin	Raw image (5 MB) → public URL in `site-media`
GET	`/admin/users`	admin	All profiles
POST	`/webhooks/clerk`	svix signature	Upsert/delete `profiles`
POST	`/webhooks/calendly`	HMAC signature	Upsert `bookings`, mark request `scheduled`; a cancel moves it back to `accepted`. 503 if signing key unset
POST	`/admin/calendly/register-webhook`	`x-admin-token`	Creates the Calendly webhook subscription (idempotent; 409 = exists)

Validation rules worth knowing: every body is parsed with zod (max lengths, 2–8 options per assessment question, `correctIndex` must point at an option); content URLs must be `https://` (http allowed only for localhost) and the Calendly URL must be on

`calendly.com`; malformed ids return 404. Missing optional integrations (Resend, Calendly, Turnstile) are logged and skipped, never a crash.

Frontend

The frontend is a React 18 single-page app built with Vite 7 and react-router 6. All text the tutor might change comes from `GET /content`, so content edits need no redeploy.

Routes

Path	Access	Page
`/`	public	Home: hero, teaching style, sample video, featured free resources, reviews
`/about`	public	Tutor bio, photo and sections
`/courses`, `/exam-prep`	public	Course catalog and exam-prep tracks
`/resources`	public	Resource library (what the viewer may see)
`/assessment`	public	Placement test; saved only when signed in
`/contact`	public	Contact/session request form (+ Turnstile when configured)
`/book?request=<id>`	public	Calendly embed for an accepted request
`/policy`	public	Policies
`/sign-in`, `/sign-up`	public	Clerk sign-in/up (`/login` redirects)
`/dashboard`	signed in	Student dashboard: next session, ready-to-book banner, requests, bookings with reschedule/cancel, courses, resources, assessments, reviews
`/profile`	signed in	Name and phone
`/admin/:tab`	admin	Tutor dashboard (tabs below)
`/admin/students/:id`	admin	One student's full record

Old paths `/students` and `/settings` redirect into `/admin`; unknown paths go to `/`.

Admin dashboard tabs

Group	Tab	What the tutor does there
Manage	Overview	Request inbox (accept/decline), booked sessions, stats
Manage	Students	Student list; click a name for their record (requests, sessions, courses + progress, assessments, private resources)
Manage	Resources	Upload files or add links/videos, set visibility, share privately, feature on home page
Website	Home page	Headline, sub-headline, teaching style, sample video link
Website	About me	Title, intro, photo (upload or link), sections
Website	Courses	Course catalog and course categories
Website	Exam prep	Intro, timelines text, exam tracks, urgency windows
Website	Assessment	Placement questions (2–8 options, correct answer, topic)
Website	Reviews	Approve, edit or delete reviews
Website	Policy	Intro and policy sections
Website	Contact & booking	Calendly event link, page intros, public email/phone, dropdown options

Admins also see an "Edit this page" button on public pages (`AdminEditLink`) that jumps to the matching tab.

How content flows

- `ContentProvider` loads `/content` once and merges it over empty defaults, so a missing key never crashes a page.
- `PageEditor` edits one page of `pages` , tracks unsaved changes and saves with `PUT /content` .
- Backend defaults (`DEFAULT_PAGES` , `DEFAULT_ASSESSMENT_QUESTIONS` in `backend/src/lib/content.ts`) fill any field never saved.

- The Calendly link saved in the admin overrides the build-time `VITE_CALENDLY_URL`.
- `vite.config.ts` writes `robots.txt`, `sitemap.xml` and absolute Open Graph image URLs from `VITE_SITE_URL`.

Key user flows

Five flows carry the business; each is covered by the end-to-end test suite.

1. Request → accept → book (the core flow)

1. A visitor or student fills in `/contact` and submits (Turnstile check if configured). `POST /requests` stores it as `new`, attaches the student id if signed in, and emails the tutor at `ADMIN_EMAIL` ("New tutoring request from …").
2. The tutor sees it in **Admin → Overview → Request Inbox** and clicks Accept or Decline (`PATCH /requests/:id`).
3. On accept, the student is emailed "Your tutoring request was accepted" with a link to `/book?request=<id>`; a signed-in student also sees a "Ready to book" banner on the dashboard.
4. `/book` shows the Calendly scheduler and passes the request id to Calendly as `utm_content`.
5. When the student books, Calendly calls `POST /webhooks/calendly` (`invitee.created`). The backend stores the booking with its cancel/reschedule links and marks the request `scheduled`.
6. The booking appears in the student's dashboard (with Reschedule / Cancel buttons that open Calendly) and in Admin → Booked sessions.
7. If the student cancels, `invitee.canceled` marks the booking `canceled` and moves the request back to `accepted`, so they can rebook. A reschedule arrives as cancel + new booking and is linked up.

Calendly sends its own confirmation emails; TutorPro sends only the two request emails above.

2. Resource library

1. Admin → Resources: upload a file (raw bytes to `POST /resources/files`, then `POST /resources`) or add a link/video.

2. Pick visibility: **Public** (anyone, listed on `/resources`; tick *Featured* to show on the home page), **Students** (any signed-in user) or **Private** (only picked students).
3. Students newly added to a private resource get "New study material: …" by email linking to their dashboard.
4. Opening a resource calls `GET /resources/:id/download`, which re-checks access and returns a 5-minute signed link (files) or the target URL (links/videos).

3. Placement assessment

The tutor edits questions in Admin → Assessment (5 built-in defaults until saved). Anyone can take the test at `/assessment`; signed-in students' results are saved (`POST /assessments`) and appear in their dashboard and in the tutor's student record.

4. Courses and progress

Students register for catalog courses from the dashboard, or the tutor assigns them from the student record. The tutor sets progress (registered → in-progress → passed); either side can drop a course.

5. Reviews and site content

Students submit reviews that wait as `pending` until the tutor approves them in Admin → Reviews. Every page's text, the tutor photo, courses, policies, dropdown options and the Calendly link are edited in the Website tabs and go live on save.

Configuration

Secrets live only in Railway; Vercel holds only public `VITE_*` values that are baked into the browser bundle — never put a secret in a `VITE_*` variable. Templates with comments: `backend/.env.example`, `frontend/.env.example`. No secret values appear in this document.

Backend (Railway → tutorpro-backend → Variables)

Variable	Purpose	Production (checked 2026-10-31)
`PORT`	Listen port	Injected by Railway — leave unset

Variable	Purpose	Production (checked 2026-10-31)
`FRONTEND_URL`	Allowed CORS origins (comma-separated); first one is used for links in emails; also Clerk authorized parties	`https://tutoring-website-ram-frontend.vercel.app`
`PUBLIC_API_URL`	Backend's own origin, for the Calendly callback	`https://tutorpro-backend-production.up.railway.app`
`CLERK_PUBLISHABLE_KEY`, `CLERK_SECRET_KEY`	Clerk server SDK	Set — **development instance** (`pk_test_` / `sk_test_`)
`CLERK_WEBHOOK_SIGNING_SECRET`	Verifies `/webhooks/clerk`	**Not set** (profiles still sync via `GET /me`)
`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`	Database + storage (service role)	Set
`RESEND_API_KEY`	Email sending	Set
`MAIL_FROM`	Sender address	`TutorPro <onboarding@resend.dev>` — Resend's test sender, **only delivers to the Resend account owner**
`ADMIN_EMAIL`	Tutor inbox for new-request emails	**Not set** — new-request emails are skipped
`CALENDLY_WEBHOOK_SIGNING_KEY`	Verifies `/webhooks/calendly` (you generate it: `openssl rand -hex 32`)	Set
`CALENDLY_PERSONAL_ACCESS_TOKEN`	Only for registering the webhook	Not set (only needed when (re)registering)
`ADMIN_API_TOKEN`	`x-admin-token` for `/admin/calendly/*` (`openssl rand -hex 32`)	Set
`TURNSTILE_SECRET_KEY`	Optional contact-form bot check	Not set
`DEV_AUTH`	Local dev sign-in	Must never be set in production

Frontend (Vercel → project → Settings → Environment Variables, Production + Preview)

Variable	Purpose
`VITE_CLERK_PUBLISHABLE_KEY`	Clerk frontend key (same Clerk app as the backend)
`VITE_API_URL`	Backend origin (`https://tutorpro-backend-production.up.railway.app`)
`VITE_CALENDLY_URL`	Fallback Calendly link; the one saved in Admin → Contact & booking wins
`VITE_SITE_URL`	Public site origin for link previews, `robots.txt` and `sitemap.xml` — recommended, set it to the live origin
`VITE_TURNSTILE_SITE_KEY`	Optional; pair with `TURNSTILE_SECRET_KEY`
`VITE_DEV_AUTH`	Local dev only

`VITE_*` changes only take effect after a Vercel redeploy. Railway variable changes trigger a redeploy automatically.

Repository variable

`API_URL` (GitHub → Settings → Variables) overrides the backend origin the keepalive workflow pings; unset uses the Railway URL above.

Local development

The whole stack runs locally with only Node 22+, Docker and the Supabase CLI. No Clerk account is needed if you use DEV_AUTH. Keep the repo outside iCloud-synced folders (Desktop/Documents), which corrupt `.git` and `node_modules`.

```
git clone https://github.com/RamtinTutorWeb/Tutoring-Website-Ram.git
~/Code/Tutoring-Website-Ram
cd ~/Code/Tutoring-Website-Ram
npm install
supabase start                                  # local Postgres + Storage;
applies migrations + seed.sql
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env.local
npm run dev:backend                             # http://localhost:4000
npm run dev:frontend                            # http://localhost:5173
```

In `backend/.env` set `SUPABASE_URL=http://127.0.0.1:54321` and the local demo service-role key printed in `supabase/README.md` (it is the Supabase CLI's public demo key, not a secret).

Option A — no Clerk (fastest): backend `DEV_AUTH=1` with Clerk keys empty; frontend `VITE_DEV_AUTH=1` with `VITE_CLERK_PUBLISHABLE_KEY` empty. `/sign-in` then shows "Tutor (admin)" and "Student" buttons for the seeded users.

Option B — real Clerk: put the Clerk development instance keys in both `.env` files and leave the `DEV_AUTH` variables empty.

Resend, Calendly and Turnstile are optional locally; without keys the API logs and skips them. Useful commands: `supabase db reset` (wipe + re-seed), `supabase stop`, Supabase Studio at `http://127.0.0.1:54323`.

Testing

There are three layers: 82 backend unit tests, 9 browser end-to-end scenarios, and a manual production test. All automated tests passed on the deployed commit `33d7ffc`. There is no CI test run on pull requests yet — run them locally before merging.

Automated

Command	What it checks	Needs
`npm run typecheck`	TypeScript, frontend + backend	nothing

Command	What it checks	Needs
`npm test`	82 vitest + supertest tests: auth/roles, DEV_AUTH guards, requests + rate limit + Turnstile, reviews moderation, content validation, resources visibility/upload/signed URLs, media upload, Clerk + Calendly webhooks (signatures, idempotency, reschedule/cancel)	nothing (in-memory fakes in `backend/test/fakes.ts`)
`npm run build`	Production builds of both packages	nothing
`npm run e2e`	Playwright in your installed Chrome against the real backend (port 4100), the Vite app (port 5180) and local Supabase, signed in via DEV_AUTH. **Resets the local database first.**	`supabase start` running, Chrome installed

The 9 e2e scenarios in `e2e/tutor-app.spec.ts`:

1. Public site works signed out (home, courses, resources, assessment).
2. Booking loop: request → admin accepts → signed Calendly webhook booking → cancel reopens the request.
3. Resources: admin uploads a private file, the student downloads it, visitors can't.
4. Admin edits Home and About pages, including photo upload, and they show on the public site.
5. Edited assessment is what students take; admin sees the result.
6. Admin assigns a course and shares material from the student record.
7. Student review is moderated; admin course edits go live.
8. Admin edits the policy page.
9. Access control: students can't reach admin pages or admin APIs.

On failure Playwright keeps a trace and screenshot under `test-results/`.

Manual production test (do once after handover, and after any integration change)

Use two accounts: the tutor (admin) and a separate student email, e.g. in a private window.

- [ ] `GET https://tutorpro-backend-production.up.railway.app/health` — every `configured` flag you expect is `true`.
- [ ] Tutor: in Admin add a course, fill About me with a photo, upload a Public resource, save the Calendly link.
- [ ] Signed out: home, about, courses and resources show the new content.
- [ ] Student: sign up, take the assessment, send a request from `/contact`.
- [ ] Tutor receives the new-request email (requires `ADMIN_EMAIL` and a verified Resend domain).
- [ ] Tutor accepts; student receives the accepted email with the booking link.
- [ ] Student books through Calendly; the booking appears in the student dashboard and Admin → Booked sessions (requires the Calendly webhook to be registered).
- [ ] Student cancels via the dashboard link; the booking shows canceled and the request is bookable again.
- [ ] Tutor shares a private resource with the student; the student is emailed and can open it.
- [ ] Student submits a review; it appears publicly only after the tutor approves it.

Deployment

Merging to `main` deploys the frontend automatically; the backend should too, but Railway's GitHub auto-deploy did not fire after the 2026-10-21 Railway incident, so check it after every merge. Database migrations are always applied by hand. `docs/DEPLOY.md` has first-time setup for every account.

Release procedure

1. Open a PR into `main`; run `npm run typecheck && npm test && npm run build` and, for UI changes, `npm run e2e`.
2. If the PR adds a file under `supabase/migrations/`, apply it to production **before** merging: `supabase link --project-ref dgnyvwxodajmkfaevpjn` then `supabase db push`. Write migrations additive and re-runnable so old code keeps working.
3. Merge. Vercel builds and deploys the frontend (status check on the commit).

4. Confirm Railway deployed the same commit: Railway dashboard → tutorpro-backend → Deployments, or `railway deployment list`. If it didn't, click **Redeploy**, or deploy from a clean checkout of `main`:

```
railway login
mkdir /tmp/deploy && git archive origin/main | tar -x -C /tmp/deploy && cd /tmp/deploy
railway up --ci --service tutorpro-backend --environment production
```

5. Verify: `/health` returns `ok`, and the frontend works against it.

Platform details

Platform	Build	Notes
Vercel	`vercel.json`: `npm ci --workspace frontend`, `npm run build --workspace frontend`, output `frontend/dist`	SPA rewrite to `index.html`; preview deployments get random origins not in `FRONTEND_URL`, so their API calls fail CORS — test on production or add a fixed preview alias
Railway	`railway.json`: Railpack builder, `npm run build --workspace backend`, start `npm start --workspace backend`	Healthcheck `/health` (60 s), restart on failure up to 5×; watches `backend/**`, root `package.json`, lockfile, `railway.json`. Railway warns that `railway.json` is deprecated in favour of `.railway/railway.ts`; existing files keep working until 2026-12-01 — run `railway config migrate` before then
Supabase	`supabase db push` from the repo root	Never run `seed.sql` in production. Free tier pauses after 7 days idle; the keepalive workflow prevents it
GitHub Actions	`keepalive.yml` daily at 09:17 UTC	Only workflow; no test CI

One-off integration steps

- **Calendly webhook** (needs a paid Calendly plan): create a personal access token in Calendly, set `CALENDLY_PERSONAL_ACCESS_TOKEN` on Railway, then `curl -X POST https://tutorpro-backend-production.up.railway.app/admin/calendly/register-webhook -H "x-admin-token: $ADMIN_API_TOKEN"`. 409 means it already exists. Re-run after changing the signing key or the API domain (delete the old subscription first).
- **Clerk webhook**: Clerk → Webhooks → endpoint `https://tutorpro-backend-production.up.railway.app/webhooks/clerk`, events `user.created`, `user.updated`, `user.deleted`; copy the signing secret to `CLERK_WEBHOOK_SIGNING_SECRET`.
- **Custom domain**: add it in Vercel (and `api.` subdomain in Railway); update `VITE_API_URL`, `VITE_SITE_URL`, `FRONTEND_URL`, `PUBLIC_API_URL`; move Clerk to a production instance with `pk_live_` / `sk_live_` keys; re-point the Calendly webhook.

Operations and troubleshooting

Start every investigation at `GET /health`: it shows whether the API is up and which integrations are configured, without exposing secrets. Backend logs are in Railway → tutorpro-backend → Deployments → the active deploy → Logs; skipped emails and webhook problems are logged there.

Symptom	Likely cause	Fix
Every page that loads data shows an error; API returns 500	Supabase free project paused	Restore it in the Supabase dashboard; check the keepalive workflow is running in GitHub Actions
Site loads but a new feature errors / 404s from the API	Railway didn't deploy the latest `main`	Redeploy (see Deployment)
Sign-in works but API calls return 401	`FRONTEND_URL` doesn't include the site's origin (Clerk authorized parties), or frontend and backend use different Clerk apps	Fix `FRONTEND_URL` / keys on Railway
Browser console shows a CORS error	Origin not in `FRONTEND_URL` (often a Vercel preview URL)	Test on production, or add the origin

Symptom	Likely cause	Fix
Tutor never receives new-request emails	`ADMIN_EMAIL` unset, or Resend sender is `onboarding@resend.dev`	Set `ADMIN_EMAIL`; verify a domain in Resend and change `MAIL_FROM`
Students don't get "accepted" or "new material" emails	Resend has no verified domain (test sender only reaches the account owner)	Verify a domain in Resend
Bookings made in Calendly never show in dashboards	Calendly webhook not registered, wrong signing key (503/401 in logs), or Calendly on the free plan	Register the webhook (Deployment)
Someone should be admin but sees no Admin link	Public metadata missing, or the 60 s role cache	Set `{ "role": "admin" }`, wait a minute, sign out and back in
Contact form returns 429	Rate limit, 10 requests per 15 min per IP	Wait; it resets automatically
Contact form returns 400 about the bot check	Only one of the two Turnstile keys is set	Set both or neither, and redeploy both
Photo or file upload fails	Over the limit (images 5 MB, files 50 MB) or unsupported image type	Shrink or convert the file
`git fetch` fails with "packfile too short"	iCloud offloaded part of `.git`	Clone fresh outside iCloud

Routine care: nothing is scheduled besides the keepalive. Check monthly that the keepalive is green and that Supabase usage is within the free tier. Data backup is whatever your Supabase plan includes; the free plan has no point-in-time recovery, so export important data occasionally (`supabase db dump`).

Security notes

The design keeps every secret and every database call on the backend; the browser holds only a Clerk session and public keys.

- **Single trusted client.** Only the backend talks to Supabase, using the service-role key. RLS with no policies plus revoked `anon` / `authenticated` grants means the database is unreachable from outside even if an anon key leaked.
- **Authorization on the server.** Role comes from Clerk `publicMetadata.role` on every request; `profiles.role` is display-only. Students only ever get their own rows (`ownerFilter`); hidden resources return 404 rather than 403, so their existence isn't revealed.
- **Files.** Resource files sit in a private bucket and are only reachable through 5-minute signed URLs issued after an access check (`Cache-Control: no-store`). Upload names are sanitized (`safeFileName`) and one stored file can't back two resources.
- **Webhooks.** Clerk (svix) and Calendly (HMAC) payloads are signature-checked and de-duplicated through `webhook_events`. Calendly manage links are stored only if they are https on `calendly.com`.
- **Abuse limits.** `POST /requests` is rate-limited per IP and can require Cloudflare Turnstile; reviews are rate-limited and moderated; all inputs are length-limited by zod; email templates HTML-escape user input.
- **Headers.** Vercel sends `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `X-Frame-Options: SAMEORIGIN` and a restrictive `Permissions-Policy`.
- **DEV_AUTH** is triple-gated on the backend and compiled out of production frontend builds.
- **Secret hygiene.** Never commit `.env` files (git-ignored) and never put a secret in a `VITE_*` variable. The only key in the repo is the Supabase CLI's public local demo key.
- **Open action: rotate keys.** Railway reported a platform security incident on 2026-10-21 and advised customers to rotate secrets stored with them and review logs. Rotate the backend secrets listed in the Offboarding checklist.

Known issues and next steps

The code has no known functional bugs; the gaps are production configuration that only an account owner can close, listed here in priority order.

Production gaps (fix before real students use it)

- [ ] **Email domain.** Verify a domain in Resend (DNS records), then set `MAIL_FROM` to e.g. `TutorPro <hello@yourdomain>`. Until then only the Resend account owner receives any email.
- [ ] **`ADMIN_EMAIL`.** Set it on Railway to the tutor's inbox; without it new-request notifications are skipped.
- [ ] **Calendly webhook.** Confirm it is registered (a test booking shows up in the dashboard); if not, register it (Deployment → One-off steps). Requires a paid Calendly plan.
- [ ] **Clerk production instance.** Production runs on Clerk's development instance (`pk_test_`), which shows a "development mode" badge and has lower limits. Move to a production instance when a custom domain exists.
- [ ] **Clerk webhook** (optional): set it up so profile edits and deletions in Clerk sync immediately.
- [ ] **`VITE_SITE_URL`** on Vercel for proper link previews and sitemap.
- [ ] **Rotate secrets** after the Railway incident.
- [ ] **Real content.** Courses are empty; fill About, Courses, Assessment, Policy and the Calendly link in Admin.

Maintenance items

- Migrate `railway.json` to Railway's new config format before 2026-12-01 (`railway config migrate`).
- Re-enable or verify Railway's GitHub auto-deploy; it did not fire for the last merge.
- Add a GitHub Actions workflow running typecheck, unit tests and build on every PR.
- `frontend/src/router` declares the `/resources` route twice (harmless duplicate; remove one).

Ideas not built (out of scope so far)

- Payments (e.g. Stripe, or Calendly's built-in payment collection on paid plans).
- Custom domain and branded emails.
- Automated backups beyond the Supabase plan.

Project history

Merged	PR	Change
2026-10-22	#8	Resource library, student records, editable assessment, photo upload, e2e suite — rebuilt on top of #5 (supersedes #6/#7)
2026-10-06	#5	Admin dashboard at `/admin/:tab`, editable pages, About page, brighter redesign, keepalive
2026-09-28	#2–#4	Hosting on Vercel + Railway + Supabase; build fixes
2026-09-04	#1	Platform foundation: Supabase, Clerk, Calendly, Vercel

Production was last deployed on 2026-10-25 (commit `33d7ffc`) via `railway up`, after Railway re-enabled builds for this plan.